Measuring and Exploiting Near-Duplicate Instruction Sequences in Executables

Ara Aslyan
Strike Technologies, New York, USA
aaslyan@striketechnologies.com

Ashot Harutyunyan
Yerevan State University, Machine Learning Lab, and
Institute for Informatics and Automation Problems, Armenia
harutyunyan.ashot@ysu.am

***Abstract*—Compiled binaries, especially generated ones, can contain large amounts of duplicate machine code, but the mechanisms deployed to remove it—linker identical-code folding (ICF), GCC `-fipa-icf`, and the LLVM *MachineOutliner*—primarily exploit byte- or register-identical code and cannot parameterize operand-differing near-duplicates. We show empirically that the dominant duplication in real, production-optimized binaries is *near*-duplicate: functions with the same instruction skeleton that differ only in operands, registers, or small blocks. On a code-generated C++ toolkit, 33–39% of functions in a binary belong to near-duplicate families. On the same generated translation unit, the LLVM MachineOutliner removes 3.87% at `-O3` and 0% at `-Oz`, whereas a parameterizing source-level merge of one family removes 22.7%—roughly 6× more and additive to the outliner. We contribute (i) a measurement of how much near-duplicate code real binaries ship and of what kind; (ii) a document-frequency discriminator that separates likely copy-paste from statically-linked library reuse without source; (iii) a profitability model (interface width × repetition × hotness) deciding which merges pay; (iv) a recursive, *fuzzy-interface* formulation that decomposes near-duplicate regions into a shared skeleton and reusable outlined "holes"; and (v) a realized, behavior-preserving transformation that reduces the binary's code section by 2.81% from a single family. Our finding is not a better outliner but a diagnosis: production tool-chains leave most operand-differing near-duplication on the table, and for code-generated software the right place to remove it is the generator.**

***Index Terms*—binary analysis, code size, function merging, outlining, code clones, code generation, sequence alignment**

I. Introduction

Code size is a first-order concern: it drives instruction-cache and iTLB pressure, program load time, memory footprint on constrained devices, and even attack surface. Tool-chains therefore deduplicate code aggressively. Linkers fold byte-identical functions via identical-code folding (ICF); GCC's `-fipa-icf` folds functions with identical bodies; and the LLVM *MachineOutliner* [1] extracts repeated identical instruction sequences into shared procedures. A common property unites them: they act on code that is *exactly* identical (in bytes, in body, or in register-precise instruction runs).

Real binaries, however, are dominated by code that is *almost* the same. Consider a routine emitted once per record type by a code generator: each copy loads the same kind of field, calls the same kind of helper, and returns the same way, differing only in a type tag, a string literal, a structure offset, or which register the allocator happened to choose. After compilation these become distinct functions with an identical instruction *skeleton* and a handful of differing operands. Exact folding cannot merge them—the bytes differ—so every copy ships in the binary.

The remedy is well known in principle: merge the copies into one function and pass the differences as parameters. *Function Merging by Sequence Alignment* [2] and its faster successors [3], [4] do exactly this—align two functions and insert parameters or selects for the mismatches. But these are research passes and are not enabled in production `-O2`/`-O3`. The mechanism exists; whether it is reaching real binaries is a separate, empirical question.

That is the question this paper answers: *how much near-duplicate code do real, production-optimized binaries actually ship, what kind is it, and is it worth merging?* We treat it as a measurement and diagnosis problem rather than as a new optimizer. Working from the opcode (mnemonic) stream recovered by disassembly, we detect near-duplicate regions with sequence alignment, separate likely copy-paste from library reuse using document frequency, classify duplication into a small taxonomy, quantify how much each deployed mechanism leaves behind, and finally realize a behavior-preserving transformation to confirm the recoverable size on a real binary and to compare head-to-head against the production outliner.

Contributions. This paper contributes: (i) a measurement of near-duplicate machine code in real optimized binaries and a three-tier taxonomy of duplicate structure (Sec. III, V); (ii) a document-frequency discriminator that separates local duplication from statically-linked library/runtime reuse without source (Sec. IV); (iii) a profitability and feasibility model based on liveness-derived interface width, repetition count, and hotness (Sec. VI); (iv) a recursive fuzzy-interface formulation that decomposes near-duplicate regions into skeletons and reusable holes (Sec. VII); and (v) a realized, behavior-preserving source-level transformation that reduces a production binary's code section by 2.81% from one generated family and exposes a ≈6× gap over LLVM MachineOutliner on the same code (Sec. VIII).

II. Background and Related Work

Exact deduplication. ICF (in `gold`/`lld`) and GCC `-fipa-icf` fold byte- or body-identical functions; `-fipa-icf` is on by default at `-O2`. The LLVM MachineOutliner [1] scans the post-register-allocation

instruction stream for repeated identical sequences and outlines them; LLVM `MergeFunctions` [5] merges functions that are identical after a structural normalization. All require identical or structurally identical instruction sequences and do not turn operand differences into explicit parameters—which, as we show, is precisely where most real near-duplication lives.

Parameterizing merge. Function Merging by Sequence Alignment [2] aligns two functions' instruction sequences, keeps the matched instructions shared, and inserts parameters and a selector for the mismatches; HyFM [3] and F3M [4] reduce its cost with locality-sensitive hashing of function fingerprints. This line establishes the *mechanism* we rely on. It does not, however, measure how much exploitable near-duplication ships in real binaries, separate copy-paste from library reuse, target code generators, or formulate recursive hole-sharing—our contributions. We use the same alignment primitive [6] as an analysis tool.

Correctness. Translation validation [7] and verified compilation [8] frame how to prove such transformations equivalence-preserving. We adopt per-instance validation, observing that our feasibility analysis already discharges the central proof obligation.

Opcode statistics and binary similarity. Opcode distributions have been used for malware detection and cross-domain sequence analysis [9], [10]; learned instruction embeddings and function-similarity systems [11], [12] and the "naturalness" of software [13] establish that assembly is statistically regular. We turn that regularity toward a size-optimization diagnosis rather than classification or search.

III. Method

Extraction. We disassemble with `objdump` and retain the mnemonic-only opcode stream per function, $s = (o_1, \ldots, o_N)$. Dropping operands erases register- and immediate-level noise, so two copies differing only in operands become *identical* opcode streams and align exactly, while genuinely different code remains distinguishable.

Candidate generation and refinement. For scalability we shingle each function into opcode k-grams and index them with MinHash+LSH [14]; candidate pairs are refined with Smith–Waterman local alignment [6], and families are connected components of the resulting similarity graph.

Taxonomy. We classify duplication by *what differs* between family members: *Tier 1*—byte-identical (linker-foldable); *Tier 2*—opcode-identical, operands differ (mergeable by promoting the differing operands to *value* parameters); *Tier 3*—near-duplicate, whole blocks differ (mergeable into a shared *skeleton* plus *code* parameters, i.e. outlined holes). Orthogonally, by document frequency across a corpus, a region is library/runtime (high frequency), generator-induced, or copy-paste-like (low frequency).

Operand parameterization (Tier 2). For an opcode-identical family we align members position-by-position and, at each position, compare operands. Positions whose operands are constant across all members remain in the body; positions that vary become parameter candidates. Crucially, many mechanically varying operands *covary*: e.g. for a 77-member accessor family in our subject, 13 varying operand slots collapse to only 4 independent parameters—a descriptor pointer driving 10 slots, two small immediates, and one callee—because slots that take a distinct value per member induce the same member partition and are therefore driven by a single underlying parameter.

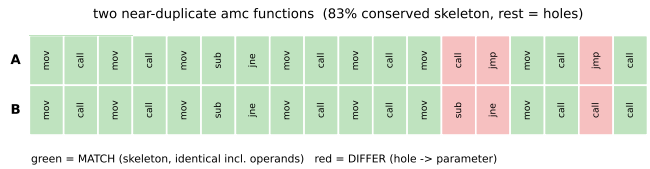

Fig. 1. Skeleton/holes on two real near-duplicate `amc` functions. Each column is an aligned position; green columns MATCH (the shared skeleton, identical including operands), red columns DIFFER (holes, supplied per member as parameters). Here 83% of columns are skeleton; the rest are cut into holes.

Skeleton/holes decomposition (Tier 3). For a Tier-3 family we star-align members to the longest member; conserved columns form the *skeleton* and maximal runs of divergent columns are *holes*. For each hole we compute its register/flags interface by local liveness on the reference function: *inputs* are upward-exposed uses (values read before being written inside the hole) and *outputs* are hole-defined values read after the hole. Fig. 1 illustrates the decomposition on two real functions: the MATCH columns (identical instructions, operands included) are the skeleton and the DIFFER columns are holes, supplied per member as parameters. The skeleton is identical across members by construction, so instantiating it with a member's arguments reproduces that member exactly; correctness follows from this rather than from any approximation (Sec. IX).

IV. Detection and Discrimination

Detection (controlled ground truth). We compiled a C source containing a function together with a verbatim copy, an edited copy, and a register-reallocated copy, plus a hard negative (a different algorithm with the same loop shape). At `-O2` opcode alignment separates every copy pair from non-copies (similarity ≥ 0.63 vs ≤ 0.20 for gcc; ≥ 0.27 vs ≤ 0.11 for clang) and detects $6/6$ copies including the edited and reallocated ones, whereas exact opcode-string matching detects only $3/6$. Two limits emerged: at `-O0` generic stack scaffolding inflates the similarity of unrelated functions and confounds detection, and cross-compiler similarity is low (0.03–0.27). Opcode-level detection is therefore reliable *within one build configuration*—the realistic case for copy-paste inside one project built one way.

Discrimination. Two binaries that statically link the same library share a large block of *identical* code, indistinguishable from copy-paste by similarity score alone. Document frequency (DF) separates them (Fig. 2). Over a 27-binary `/usr/bin` corpus (394,358 distinct 12-grams, with only a tiny universal core), an injected ground-truth copy-paste appears in exactly its two binaries (63 exclusive shingles), real shared `gnulib` code

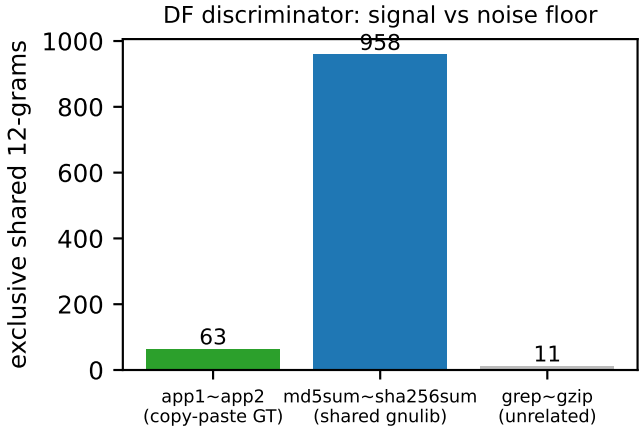

Fig. 2. Document-frequency discriminator. Shingles shared *exclusively* by a binary pair: injected copy-paste and shared library both "light up" well above the unrelated-pair noise floor; document frequency across the corpus (not shown here) is what separates the first two.

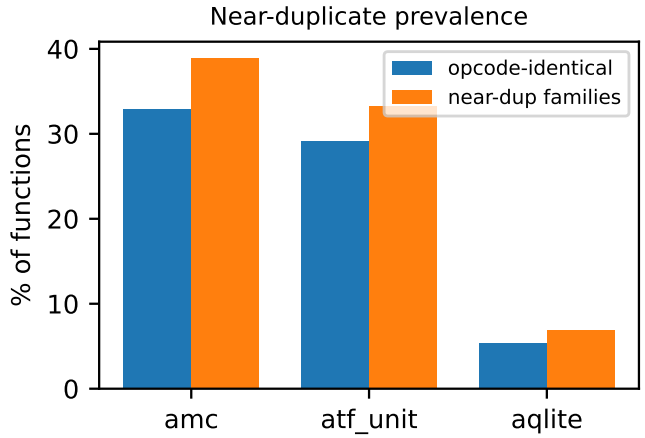

Fig. 3. Near-duplicate prevalence per binary. The rate tracks the fraction of each binary that is code-generated (`aqlite` is mostly hand-written).

TABLE I
NEAR-DUPLICATE FUNCTION PREVALENCE (OPCODE-ONLY, FUNCTIONS $\geq$ 12 INSTR., JACCARD $\geq$ 0.85).

Binary	Functions	Opcode-identical	Near-dup families
`amc`	2022	32.9%	38.9%
atf_unit	1318	29.1%	33.2%
aqlite	1624	5.4%	6.9%

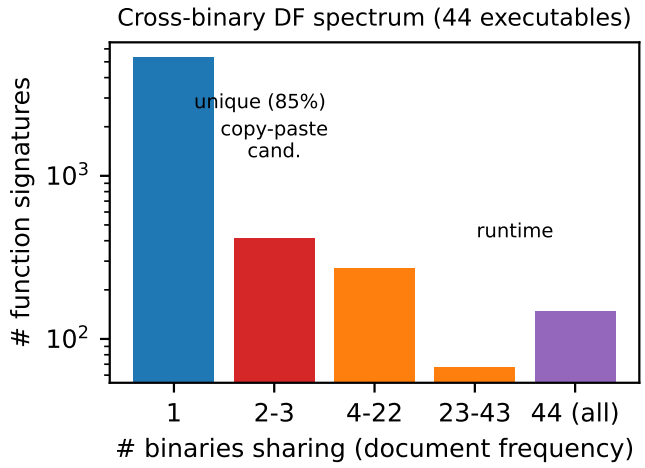

Fig. 4. Cross-binary document-frequency spectrum over 44 executables (log scale). Most code is unique; a thin band is narrowly shared (copy-paste candidates); a small core is the runtime linked into every tool.

between `md5sum` and `sha256sum` appears as 958 exclusive shingles, and an unrelated pair shares only 11 (the noise floor). Library reuse is high-DF; copy-paste is low-DF; similarity score is the wrong axis, *frequency* is the right one.

V. Prevalence and Recoverable Opportunity

We study a non-stripped, code-generated C++ toolkit (OpenACR), whose `amc` generator emits per-type accessor functions; this makes it an ideal, ground-truthed subject for generator-induced duplication.

Intra-binary. Table I and Fig. 3 show that 33–39% of functions in generated binaries belong to near-duplicate families. `aqlite`, which is mostly hand-written external SQLite code, is an order of magnitude lower, so the near-duplicate rate tracks how much of a binary is machine-generated rather than being an artifact of our method. The analysis is implemented as a single reusable tool that takes arbitrary binaries; run across 42 executables, opcode-identical prevalence ranges from 5.4% (hand-written `aqlite`) to 32.6% (`amc`), with mergeable opportunity from a few KB up to 113 KB per binary.

Cross-binary. Across 44 executables, of 6,189 distinct function signatures, 5,291 (85%) are unique to a single binary, 413 are shared by 2–3 binaries (copy-paste / narrow-sharing candidates), and 148 appear in *all* 44—the statically-linked runtime (Fig. 4). DF cleanly stratifies the corpus into tool-specific code, narrowly-shared code, and ubiquitous runtime.

What exact mechanisms leave behind. On `amc`, 608 functions (31%) are opcode-identical duplicates, yet only 3 are byte-identical, so ICF and `-fipa-icf` reach about 0.1% of `.text`. Promoting operands to parameters (Tier 2) instead exposes an estimated 10.6% (≈112 KB). On a single generated translation unit, three accessor families (`ReadStrptrMaybe`, `ReadFieldMaybe`, and getters) account for ≈106 KB of near-duplicate code.

VI. Profitability and Feasibility

Merging pays only when the shared body dominates the per-call overhead. For a family of N members, a hole of size s instructions, and a per-call-site interface overhead $o \approx (\#\text{inputs} + \#\text{outputs} + 2)$, the net saving is

$$\Delta = (N-1)\,s \;-\; N\,o. \tag{1}$$

Interface width is computed by liveness: a hole is a clean outline target when few values cross its boundary. The same quantity is a *correctness* side condition—the interface must

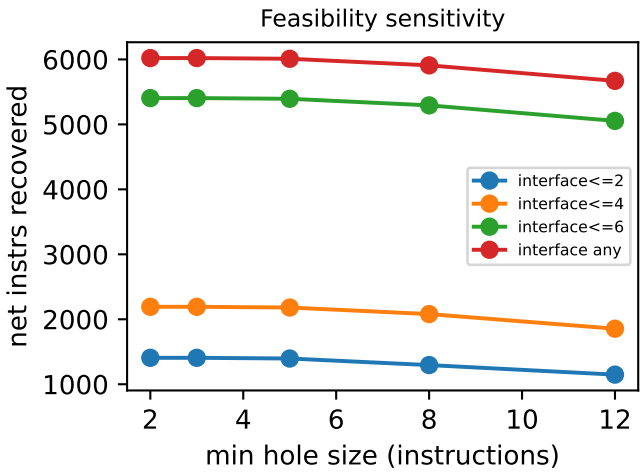

Fig. 5. Feasibility sensitivity. Recoverable net instructions versus minimum hole size and maximum interface width: interface width dominates; the hole-size floor is secondary.

capture *all* crossing values, including processor flags—so the feasibility analysis doubles as a soundness check (Sec. IX).

Across 40 Tier-3 families in `amc`, the largest (a 15-member insert routine) is 72% conserved skeleton with three sizeable holes; over all families the holes carry 11,204 instructions of reduction that is unreachable by exact or value-only merge. Of 84 large holes, 92% have a narrow interface; under the net-saving model 61% (51/84) are profitable. Notably, 4 of 7 *wide*-interface holes remain profitable because high repetition amortizes a wide interface (e.g. N=12, 68 instructions, 5 inputs/4 outputs $\Rightarrow$ net +616 instructions). Interface width alone is the wrong gate; net saving—width *times* repetition—is the right one, modulated by hotness (cold, repetitive, wide holes still merge well; hot inner loops prefer inlining).

Feasibility is governed by a small set of explicit parameters—minimum hole size, maximum tolerated interface width, the overhead model, and minimum repetition—rather than fixed constants. Fig. 5 sweeps the two principal ones. The maximum interface width is the dominant lever: relaxing it from ≤ 4 to ≤ 6 crossing values more than doubles the recoverable instructions, whereas raising the hole-size floor from 2 to 12 costs only ≈6%. The binding constraint on profitable merging is the data-flow interface, not the size of the differing block.

VII. Recursive Fuzzy-Interface Merging

A skeleton/holes decomposition shares the skeleton once. But holes are themselves code fragments that recur *across* families, so we recurse: pool all holes and deduplicate them. Once a hole is an outlined function rather than an inlined fragment, register naming becomes irrelevant—the interface is a *signature shape*—so two holes that differ only by register allocation are the same function. We therefore cluster holes by *fuzzy interface*: holes whose opcode content aligns (≥ 0.85) and whose interfaces differ by at most one input/output are unified, widening the shared signature (and ignoring the surplus argument) where needed.

The effect is decisive. Exact hole-sharing is negligible: 7 holes in 3 clusters, 14 instructions saved. Fuzzy-interface clustering collapses the 84 holes to 55 clusters and saves 1,062 second-order instructions—75× the exact approach—with 12 of 13 unifying clusters *requiring* interface widening. Recursion composes because observational equivalence is a congruence; widening additionally needs a dead-argument soundness lemma (the surplus parameter must be unused in the narrower instantiation). The fuzzy/widening step, not exact matching, is what unlocks second-order reuse.

TABLE II
Realized code-size reduction and comparison to production mechanisms (real `dmmeta_gen`, clang `-O3` unless noted).

Configuration	.text (B)	removed
baseline (clang -O3)	110,111	—
+ LLVM MachineOutliner	105,854	3.87%
our merge (one family)	85,141	**22.68%**
merge + MachineOutliner	80,996	26.45%
clang -Oz, no outliner	95,910	—
clang -Oz, + MachineOutliner	95,910	0.00%
clang -Oz, our merge + outliner	77,244	19.46%

VIII. Realized Transformation and Baseline Comparison

We realize the Tier-2 merge on real `amc` source. The `ReadStrptrMaybe` family is maximally regular: each member differs only in its type, two string literals, and a per-type callee. A source rewrite replaces each member with a thin thunk to one type-erased shared helper:

```
static bool Rsm_shared(void* p, strptr in,
   const char* lc, const char* uc, RsmRF rf){
 bool r=true;
 r = StripTypeTag(in,lc)||StripTypeTag(in,uc);
 ind_beg(Attr_curs,a,in){ r=r&&rf(p,a.name,a.value);}
    ind_end;
 return r; }
// per type T:
bool T_ReadStrptrMaybe(T& parent, strptr in){
 return Rsm_shared(&parent,in,"t.lc","t.uc",(RsmRF)
    T_ReadFieldMaybe); }
```

The type-erased function-pointer cast is legal under the project's own `-Wno-cast-function-type`. We reproduce the exact production build command (`gcc -O3`); our baseline object matches the shipped object byte-for-byte (159,980 B `.text`), confirming a faithful reproduction. Because MachineOutliner is an LLVM facility, the comparison in Table II is run under `clang`, and its percentages are over the *translation unit's* `.text` (clang's `-O3` baseline for this unit is 110,111 B); the whole-binary figure below is over `amc`'s `.text`.

Comparison. On identical real code (Table II, Fig. 6), the LLVM MachineOutliner removes 3.87% of the translation unit at `-O3` and 0% at `-Oz`: it folds register-identical runs but cannot parameterize the differing operands. Our merge of one family removes 22.68% of the same translation unit (≈6× more) and is *additive* to the outliner (26.45% combined), since the two target different classes. We also tested the strongest

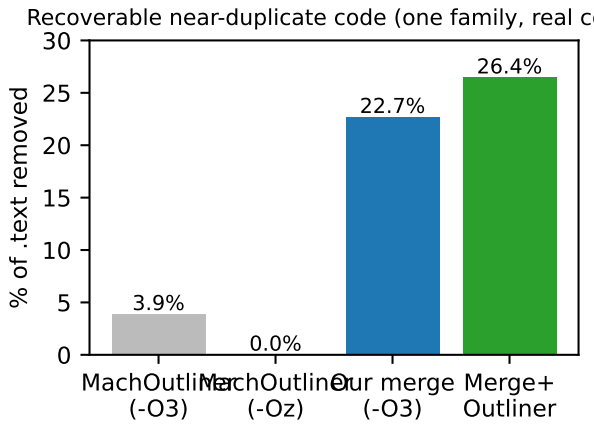

Fig. 6. What each mechanism recovers on the same real code (one family). The production outliner reaches $\leq$ 3.87%; parameterizing merge reaches 22.7% and is additive to the outliner.

deployed IR-level mechanism, LLVM `MergeFunctions` (run explicitly via `opt`, as it is not in the default `-O2/-O3` pipeline): it removes 7.89% of the unit—more than the machine outliner, but still $\approx$2.9$\times$ less than our merge, because it folds structurally identical functions yet does not parameterize differing constants or callees. Across all three deployed layers—byte (ICF, $\approx$0.1%), IR (`MergeFunctions`, 7.89%), and machine (`MachineOutliner`, 3.87%)—operand-differing near-duplication is left largely unmerged. The shipped binary, built with `gcc -O3` (`-fipa-icf` on by default), still contained all 243 family copies, corroborating that deployed exact folding does not reach this class.

Whole-binary, behavior-preserving. Merging the family across all 10 generated translation units and rebuilding `amc` reduces its `.text` from 1,637,878 to 1,591,871 B ($-$2.81%) from a *single* family. The build is clean under `-Werror`, a read-only query (`acr ctype:%`) produces byte-identical output (1,419 lines) before and after, and the full `atf_unit` suite shows *no regressions*: stock and merged builds both run 254 tests with the same 6 pre-existing failures (unrelated to the merge), the only output differences being nondeterministic temporary-file names and timing values. A controlled microbenchmark confirms the size/speed trade-off: realistic 122-byte accessors merge to $-$30% `.text` with identical output at +8% runtime; the shared body must be marked `noinline` or `-O2` re-inlines it and reverses the merge; and below a size threshold (e.g. 55-byte bodies) merging loses on both size and speed because overhead exceeds the body.

IX. Correctness

We claim no new theorem. Correctness here is the standard notion of *observational (behavioral) equivalence*, established by the usual simulation argument from verified compilation [8] and translation validation [7], [15]: a relation pairs an original machine state with a transformed state that agree on all live, non-scratch locations, and each original step is matched while the relation is preserved. What we provide is the *specialization* of that argument to skeleton/holes merging—a set of *proof obligations*, each a restatement of a well-known requirement: (1) *skeleton identity*—conserved positions are syntactically identical (by construction of the alignment); (2) *pure operand substitution*—each operand promoted to a parameter is bound at the call site to exactly its original value; (3) *interface completeness*—each hole's interface captures every value crossing its boundary (live-in/live-out registers, flags, memory), which is precisely the live-variable analysis underlying procedural abstraction and outlining [16], [17]; (4) *ABI/frame preservation*—caller/callee-saved registers, stack alignment, and frame-relative access are preserved; and, for the recursive fuzzy-interface step (Sec. VII), (5) *dead-argument soundness*—when two holes are unified by widening to a superset signature, the surplus parameter is unused in the instantiation that did not require it.

Completeness is correctness; width is profitability. Obligation (3) requires the interface be *complete*, not narrow. The maximum-interface-width parameter of Sec. VI (Fig. 5) is a profitability filter, not a correctness condition: widening the tolerated interface only adds parameters to an already-complete interface, changing which merges pay but never their soundness. Our liveness feasibility gate computes exactly the interface of obligation (3), so the same analysis that scores profitability also discharges the central soundness check—a hole that produces flags read after it but omits them from its interface is an *incorrect* merge, not merely an expensive one.

Assurance in practice. We discharge these obligations per instance rather than once-and-for-all over a full x86 semantics: the realized merge compiles under production `-Werror` flags, passes a differential test on real data (identical query output), and introduces no regressions across the full `atf_unit` suite. A full per-instance SMT equivalence check [7] is future work, and recursion composes because observational equivalence is a congruence. Obligation (4) is platform-specific—our type-erased merge is sound on the x86-64 SysV ABI we tested and would need re-checking on another ABI.

X. Discussion

The gap between deployed mechanisms and the available opportunity is structural, not a tuning artifact: outlining and ICF require register/byte identity, whereas the dominant near-duplication differs in operands. Three deployment points are possible. A *compiler pass* (as in [2]) is general but competes with mature infrastructure and pays an indirect-call cost. *Post-link* rewriting is hardest. For code-generated software, the cheapest and safest point is the *generator*: emitting one parameterized helper removes the duplication at the source, leaves the compiler free to inline where hot, and avoids the indirect call entirely. This is why we frame the result as a diagnosis with an actionable target rather than as another optimizer.

On the size/speed question, smaller code is not strictly a trade against speed: for cold, scattered code—which generated accessors typically are—reducing the footprint improves instruction-cache and iTLB behavior and can be net-positive, while hot

inner loops favor inlining. The hotness term in the profitability model captures this, and the realized microbenchmark ($+8\%$ on a hot synthetic loop) bounds the worst case.

XI. Threats to Validity

Construct. Opcode normalization discards operands, so similarity reflects structural rather than semantic equivalence; we therefore validate merges behaviorally rather than claim semantic equivalence from the opcode stream alone. *Internal.* Most opportunity figures are estimated from disassembly; one family in one binary is fully realized and behavior-checked, and the comparison against MachineOutliner is measured on real code. *External.* Results are x86-64-specific and centered on one code-generated subject plus a controlled corpus; `-O0` confounds detection and cross-compiler detection is weak. *Provenance.* From a binary one cannot infer whether a near-duplicate arose from human copy-paste, templates, macros, or a generator; we use "copy-paste-like" for low-DF near-duplicates without asserting source-level cause.

XII. Conclusion

Production-optimized binaries ship with substantial near-duplicate machine code that deployed mechanisms do not remove: on real code the LLVM MachineOutliner reaches $\leq 3.87\%$ of one family (and 0% in size mode), while a parameterizing merge reaches 22.7%, additively. We measured how much such code exists and of what kind, separated copy-paste from library reuse without source, modeled when merging pays, gave a recursive fuzzy-interface formulation that unlocks second-order reuse, and realized a behavior-preserving 2.81% whole-binary reduction from a single family. The actionable conclusion is to remove generator-induced duplication in the generator. Future work: realize Tier-3 families via data-driven dispatch, formal per-instance validation, and multi-architecture, multi-tool-chain measurement.

References

[1] J. Paquette *et al.*, "Reducing code size using the LLVM machine outliner," in *LLVM Developers' Meeting*, 2016.

[2] R. C. O. Rocha, P. Petoumenos, Z. Wang, M. Cole, and H. Leather, "Function merging by sequence alignment," in *IEEE/ACM Int. Symp. Code Generation and Optimization (CGO)*, 2019.

[3] ——, "HyFM: Function merging for free," in *ACM SIGPLAN/SIGBED Conf. Languages, Compilers, and Tools for Embedded Systems (LCTES)*, 2020.

[4] R. C. O. Rocha *et al.*, "F3M: Fast focused function merging," in *IEEE/ACM Int. Symp. Code Generation and Optimization (CGO)*, 2021.

[5] "LLVM mergefunctions pass," LLVM Documentation, https://llvm.org/docs/MergeFunctions.html, 2024.

[6] T. F. Smith and M. S. Waterman, "Identification of common molecular subsequences," *J. Molecular Biology*, vol. 147, pp. 195–197, 1981.

[7] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr, "Alive2: Bounded translation validation for LLVM," in *ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2021.

[8] X. Leroy, "Formal verification of a realistic compiler," *Communications of the ACM*, vol. 52, no. 7, pp. 107–115, 2009.

[9] D. Bilar, "Opcodes as predictor for malware," in *Int. J. Electronic Security and Digital Forensics*, 2007.

[10] Z. Gan *et al.*, "A statistical comparison of DNA, english, source and binary sequences," *(cross-domain n-tuple Zipf analysis)*, 2009.

[11] X. Li, Y. Qu, and H. Yin, "PalmTree: Learning an assembly language model for instruction embedding," in *ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2021.

[12] A. Marcelli *et al.*, "How machine learning is solving the binary function similarity problem," in *USENIX Security Symposium*, 2022.

[13] A. Hindle, E. T. Barr, Z. Su, M. Gabel, and P. Devanbu, "On the naturalness of software," in *Int. Conf. Software Engineering (ICSE)*, 2012.

[14] A. Z. Broder, "On the resemblance and containment of documents," *Proc. Compression and Complexity of Sequences*, 1997.

[15] A. Pnueli, M. Siegel, and E. Singerman, "Translation validation," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 1998.

[16] K. D. Cooper and N. McIntosh, "Enhanced code compression for embedded RISC processors," in *ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 1999.

[17] S. K. Debray, W. Evans, R. Muth, and B. De Sutter, "Compiler techniques for code compaction," *ACM Trans. Programming Languages and Systems (TOPLAS)*, vol. 22, no. 2, pp. 378–415, 2000.